



TRAINING PROGRAMME

# Machine Learning & Deep Learning

From linear models to Transformers, with the mathematics behind them

---

**75** HOURS + 9 H OPTIONAL

**FOR** Developers, analysts and engineers who want to build machine learning models



**Mohamed Aziz BEN HAH**

[mohamedazizbenhaha.netlify.app](https://mohamedazizbenhaha.netlify.app)

## Overview

This programme teaches participants to build, evaluate and explain machine learning models, and the mathematics that makes them work. They cover supervised learning, from linear and logistic regression to trees, ensembles and support vector machines, with sound evaluation, tuning, pipelines and experiment tracking; unsupervised learning; and deep learning, from a network built from scratch to PyTorch, CNNs, attention and the Transformer. A responsible AI phase covers explainability, fairness and governance, an optional reinforcement learning module adds 9 h, and the programme closes with an end-to-end machine learning project presented by each team.

## At a glance

| #  | PHASE                  | FOCUS                                                                                    | HOURS |
|----|------------------------|------------------------------------------------------------------------------------------|-------|
| 01 | Supervised Learning    | Regression, classification, trees, ensembles, SVMs, evaluation, pipelines                | 24 h  |
| 02 | Unsupervised Learning  | Clustering, mixture models, PCA, manifold learning, anomalies, recommenders              | 12 h  |
| 03 | Deep Learning          | Neural networks from scratch, backpropagation, PyTorch, CNNs, attention and Transformers | 15 h  |
| 04 | Responsible AI         | Explainability, fairness, the EU AI Act, model cards                                     | 6 h   |
| 05 | Reinforcement Learning | <b>OPTIONAL</b> MDPs, dynamic programming, Q-learning, DQN, policy gradients             | +9 h  |
| 06 | Final Project          | An end-to-end machine learning project, presented to the group                           | 9 h   |
|    | Buffer                 | Catch-up, revision, extra lab time                                                       | 9 h   |
|    | Total                  | Not included: optional phase 05, + 9 h                                                   | 75 h  |

## Key facts

|                 |                                                                                                                                                                                                               |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AUDIENCE        | Developers, analysts and engineers who want to build machine learning models                                                                                                                                  |
| PREREQUISITES   | Python, NumPy and pandas (course: Python for Data Science), or equivalent; secondary-school algebra. The mathematics needed is taught in the course                                                           |
| DURATION        | 75 h: 66 h of chapters + 9 h buffer; optional reinforcement learning module: +9 h                                                                                                                             |
| FORMAT          | Lectures that derive each method, followed by labs in Jupyter; the mathematics is taught where it is used                                                                                                     |
| LAB ENVIRONMENT | One laptop per participant (16 GB RAM, Python 3, JupyterLab, scikit-learn, PyTorch, MLflow); GPU notebooks on Google Colab or a GPU server provided by the organiser for deep learning; no paid cloud account |
| ASSESSMENT      | A lab validated at the end of each phase and a final project presented to the group; certificate of completion for participants who complete both                                                             |
| OUTCOME         | Participants can frame a machine learning problem, build and compare classical and deep learning models, evaluate them without leakage, explain their predictions and track the experiments                   |

PHASE 01

24 H

# Supervised Learning

Participants learn how a model generalises and how to measure it honestly, then build the main supervised methods from their mathematics: linear and logistic regression trained by gradient descent, decision trees and ensembles, and support vector machines. They finish with cross-validation, hyperparameter search, feature pipelines and experiment tracking with MLflow. Validated lab: a tuned scikit-learn pipeline compared with two baselines on a held-out test set, with every run tracked in MLflow.

| #   | CHAPTER                                      | CONTENT                                                                                                     | FORMAT        | HOURS |
|-----|----------------------------------------------|-------------------------------------------------------------------------------------------------------------|---------------|-------|
| 1.1 | <b>Framing &amp; Linear Regression</b>       | Regression vs classification, generalisation, data splits, data leakage; vectors, matrices, normal equation | Lecture + Lab | 3 h   |
| 1.2 | <b>Gradient Descent &amp; Regularisation</b> | Derivatives, chain rule, gradient descent, ridge and lasso                                                  | Lecture + Lab | 3 h   |
| 1.3 | <b>Logistic Regression &amp; Metrics</b>     | Sigmoid, maximum likelihood, cross-entropy, precision, recall, ROC                                          | Lecture + Lab | 3 h   |
| 1.4 | <b>Decision Trees</b>                        | Entropy, information gain, Gini impurity, CART                                                              | Lecture + Lab | 3 h   |
| 1.5 | <b>Ensembles</b>                             | Bagging, random forests, boosting (XGBoost, LightGBM)                                                       | Lecture + Lab | 3 h   |
| 1.6 | <b>Support Vector Machines</b>               | Margins, soft margin, kernels, SVR                                                                          | Lecture + Lab | 3 h   |
| 1.7 | <b>Evaluation &amp; Tuning</b>               | Cross-validation, learning curves, grid, random and Bayesian search                                         | Lecture + Lab | 3 h   |
| 1.8 | <b>Feature Engineering &amp; Pipelines</b>   | Encoding, scaling, scikit-learn pipelines, MLflow tracking                                                  | Lecture + Lab | 3 h   |

## MATHS COVERED IN THIS PHASE

Linear algebra (vectors, matrices, normal equation) · Calculus (derivatives, chain rule, gradient) · Statistics (maximum likelihood, cross-entropy, bias and variance) · Information theory (entropy, information gain) · Optimisation (convexity, gradient descent, L1 and L2 penalties, constrained optimisation for the SVM margin, kernels)

## PHASE 02

12 H

# Unsupervised Learning

Participants find structure in data without labels: hierarchical, density-based and centroid clustering, Gaussian mixtures trained with expectation-maximisation, dimensionality reduction with PCA, t-SNE and UMAP, anomaly detection, association rules and collaborative filtering. Validated lab: a customer segmentation with K-Means and a Gaussian mixture, visualised after PCA, with the choice of the number of clusters justified.

| #   | CHAPTER                          | CONTENT                                                         | FORMAT        | HOURS |
|-----|----------------------------------|-----------------------------------------------------------------|---------------|-------|
| 2.1 | Unsupervised Framing             | Clustering, density estimation, dimensionality reduction        | Lecture + Lab | 1.5 h |
| 2.2 | Hierarchical Clustering & DBSCAN | Linkage criteria, density-based clustering                      | Lecture + Lab | 1.5 h |
| 2.3 | K-Means & Gaussian Mixtures      | Objective function, Lloyd's algorithm, expectation-maximisation | Lecture + Lab | 3 h   |
| 2.4 | PCA                              | Covariance, eigendecomposition, explained variance              | Lecture + Lab | 3 h   |
| 2.5 | Manifolds & Anomalies            | t-SNE, UMAP, isolation forest                                   | Lecture + Lab | 1.5 h |
| 2.6 | Association Rules & Recommenders | Apriori, support, confidence and lift, collaborative filtering  | Lecture + Lab | 1.5 h |

## MATHS COVERED IN THIS PHASE

Probability (multivariate Gaussian, expectation-maximisation) · Linear algebra (covariance matrix, eigendecomposition) · Distances and similarity measures

## PHASE 03

15 H

# Deep Learning

Participants build a neural network in NumPy to see every step, derive backpropagation, then train models with PyTorch: convolutional networks and transfer learning for images, recurrent networks for sequences, attention and the Transformer architecture. Validated lab: an image classifier fine-tuned from a pretrained CNN in PyTorch, with its training curves and checkpoints.

| #   | CHAPTER                       | CONTENT                                                                                | FORMAT        | HOURS |
|-----|-------------------------------|----------------------------------------------------------------------------------------|---------------|-------|
| 3.1 | Perceptron & MLP from Scratch | NumPy implementation of a multilayer perceptron                                        | Lecture + Lab | 3 h   |
| 3.2 | Backpropagation               | Computational graphs, vanishing gradients                                              | Lecture + Lab | 3 h   |
| 3.3 | Training with PyTorch         | Tensors, autograd, training loops, optimisers (SGD, Adam), regularisation, checkpoints | Lecture + Lab | 3 h   |
| 3.4 | CNNs                          | Convolution, pooling, architectures, transfer learning                                 | Lecture + Lab | 3 h   |
| 3.5 | Sequences & Attention         | RNN, LSTM and GRU, attention, the Transformer architecture                             | Lecture + Lab | 3 h   |

**MATHS COVERED IN THIS PHASE**

Calculus (chain rule on computational graphs) · Linear algebra (matrix products, convolution) · Optimisation (stochastic gradient descent, Adam) · Probability (softmax, cross-entropy)

## PHASE 04

6 H

# Responsible AI

Participants explain model predictions globally and locally, measure and mitigate bias, and document models for governance under the EU AI Act and data protection rules. Validated lab: a SHAP explanation and a fairness report for the phase 1 model, summarised in a model card.

| #   | CHAPTER                     | CONTENT                                                          | FORMAT             | HOURS |
|-----|-----------------------------|------------------------------------------------------------------|--------------------|-------|
| 4.1 | Explainability              | SHAP, LIME, feature importance                                   | Lecture + Lab      | 3 h   |
| 4.2 | Bias, Fairness & Governance | Fairness metrics, the EU AI Act and data protection, model cards | Lecture + Workshop | 3 h   |

## PHASE 05 · OPTIONAL

+9 H

# Reinforcement Learning

This optional module adds 9 h for groups who want it. Participants model decision problems as Markov decision processes, solve them with dynamic programming, then learn from experience with Q-learning and SARSA, deep Q-networks and policy-gradient methods. Validated lab: an agent trained with Q-learning and then with a DQN on a Gymnasium environment, with the learning curves compared.

| #   | CHAPTER                         | CONTENT                                                  | FORMAT        | HOURS |
|-----|---------------------------------|----------------------------------------------------------|---------------|-------|
| 5.1 | RL & MDPs                       | Agent, environment, rewards, policies, Bellman equations | Lecture + Lab | 1.5 h |
| 5.2 | Dynamic Programming             | Value iteration and policy iteration                     | Lecture + Lab | 1.5 h |
| 5.3 | Q-Learning & SARSA              | Temporal-difference learning                             | Lecture + Lab | 1.5 h |
| 5.4 | Deep Q-Networks                 | Experience replay, target networks                       | Lecture + Lab | 1.5 h |
| 5.5 | Policy Gradients & Actor-Critic | REINFORCE, A2C, PPO overview                             | Lecture + Lab | 3 h   |

## MATHS COVERED IN THIS PHASE

Probability (Markov decision processes, expectations) · Dynamic programming (Bellman equations) · Calculus (policy gradient theorem, log-derivative trick)

## PHASE 06

9 H

# Final Project

Teams of two or three carry a machine learning problem from framing to explanation on a provided dataset, reusing the phase labs, then present it as they would to a product team.

| #   | CHAPTER                          | CONTENT                                                                                           | FORMAT  | HOURS        |
|-----|----------------------------------|---------------------------------------------------------------------------------------------------|---------|--------------|
| 6.1 | <b>Framing &amp; Exploration</b> | Frame the problem and the metric, exploratory data analysis, split strategy                       | Project | <b>3 h</b>   |
| 6.2 | <b>Models &amp; Experiments</b>  | Feature pipeline, comparison of classical models and a deep learning model, all tracked in MLflow | Project | <b>3 h</b>   |
| 6.3 | <b>Explain &amp; Document</b>    | Explainability report, model card, short technical report                                         | Project | <b>1.5 h</b> |
| 6.4 | <b>Presentation &amp; Review</b> | Presentation of the results and limits; questions and feedback                                    | Project | <b>1.5 h</b> |

## FINAL PROJECT

## End-to-End Machine Learning Project

A machine learning problem carried from framing and exploration to a feature pipeline, a comparison of classical models with a deep learning model, an explainability report and a model card, with every experiment tracked.

**STACK** Python, pandas, scikit-learn, XGBoost or LightGBM, PyTorch, MLflow, SHAP, JupyterLab

**DELIVERABLE** Repository with notebooks and pipeline code, MLflow runs, explainability report, model card, presentation